Intelligent Capability Networks Capability Coordination and Trust

Formal Reasoning about Authority Attenuation Contract Composition and Outcome Evidence

中文 · Reading guide

Abstract

Discovering a capability does not establish a caller's right to use it, and accepting an action does not establish that its goal has been achieved. Coordination in an intelligent capability network must therefore address behavioral contracts, authority, time validity, and outcome evidence together. This paper proposes organizing coordination around proof obligations. A candidate capability carries distinct obligations for compatibility, execution permission, state validity, and goal satisfaction; a result claim covers only the portion supported by evidence.

We model authority attenuation along delegation paths and show that a composite task's authority requirements combine action demands by union while remaining constrained by superior grants. We develop sequential composition and concurrent noninterference conditions for relational contracts, derive a stale-authorization window from revocation propagation, credential lifetime, and the delay of already accepted actions, and use indistinguishable executions to show the conflict between immediate revocation and full authorization availability during partitions. For outcome verification, we distinguish authentic observations from true goals, derive conservative error bounds for correlated verifiers, and give a dependency rule for propagating evidence invalidation.

The research proposal is to carry unresolved proof obligations with capability coordination and connect discovery coverage evidence to execution outcome evidence. Contract logic, least privilege, authority attenuation, and probabilistic reasoning have established foundations. These methods are not renamed as new theorems, and the plausibility of the framework is not treated as experimentally verified effectiveness.

Keywords: intelligent capability networks; capability contracts; authority attenuation; coordination correctness; revocation timing; outcome evidence

1 From capability discovery to accountable coordination

ICN here means Intelligent Capability Network. A capability is a contract for a class of state transitions under specified conditions, rather than a functional description or endpoint address. The network may include services, models, sensors, and actors that change the physical environment. An administrative domain is accountable for its outward declarations and authorization boundaries.

Coordination involves initiators, capability owners, discoverers, executors, and outcome verifiers. One entity may perform several roles, but combining roles does not eliminate their proof obligations. A discoverer that locates a temperature-measurement service does not thereby become a trusted source of temperature observations. An agent authorized to find a service on a user's behalf does not thereby acquire authority to activate equipment.

The companion paper Recursive Capability Discovery under Policy Constraints studies coverage, budgets, and consistency of candidate declaration sets. This paper begins with a returned candidate and asks what it may replace, who permits which actions, when that permission remains valid, and what evidence supports goal satisfaction. Both papers use explicit reference scopes to prevent local conclusions from becoming system-wide guarantees.

The official MCP and A2A specifications provide interaction semantics for tools and agents [1–2]; their existence does not establish any business goal. NIST's zero trust architecture provides background for evaluating access by resource and request [3]. Behavioral subtyping, Hoare logic, and constrained delegation credentials provide established foundations for contracts, composition, and authority attenuation [4–6]. The proposed increment concerns keeping these conditions consistent within a cross-domain task and limiting result claims when evidence is insufficient.

2 Capability model and proof obligations

2.1 State relations and action authority

Let S be the relevant world-state space and $\mathcal{A}$ the set of concrete actions qualified by resource, principal, operation, purpose, and temporal context. Capability c has behavioral specification

$$K_c = (P_c, R_c, Q_c, E_c), \qquad R_c \subseteq S \times S. \qquad (C1)$$

Here P_c is the set of pre-states, R_c the permitted transition relation, Q_c the post-state set, and $E_c \subseteq \mathcal{A}$ the permitted actions or effects. Inputs, outputs, and environmental observations can be included in the state, so an output field cannot conceal undeclared physical effects. For actual transitions from valid pre-states, partial correctness requires

$$R_c(P_c) \subseteq Q_c, \qquad \mathrm{Effects}(c) \subseteq E_c. \qquad (C2)$$

$R_c(P_c)$ denotes the successors reachable from P_c. C2 proves neither termination nor a deadline: a capability that never finishes may still satisfy partial correctness. Tasks requiring termination need a separate total-correctness or bounded-completion obligation.

2.2 Four obligations instead of one trust score

For a capability-use request x, define a compatibility obligation O_T, an authorization obligation O_A, a state and freshness obligation O_F, and an outcome-evidence obligation O_G. These have

different premises and evidence sources. An obligation is supported, refuted, or unresolved; a weighted trust score must not compensate for missing permission.

Discovery can at most support some contract-matching claims. Before committing an action, O_A and the relevant O_F must be supported. A completion claim requires support for the corresponding O_G. This staging is a semantic constraint of the proposed framework. A record marked supported does not make an obligation true: its evidence must meet the checkable conditions specified below.

2.3 Common assumptions

All propositions concern explicit action scopes and reference states. Authorization predicates are decidable, authoritative identities and grant origins are verifiable, and relevant effects cannot bypass the modeled authorization decision. Actual compliance with a relational contract is an external proof or verification obligation, not something established by a provider's self-description. Physical effects require a state model with explicit observability, measurement errors, and environmental interference. Missing premises weaken conclusions to statements about declarations or observations.

3 Attenuation and composition of delegated authority

3.1 A single delegation path

Fix the decision time and complete context. Let $A_0 \subseteq \mathcal{A}$ be a root grant and $P_j \subseteq \mathcal{A}$ the restriction added by delegation step j. The child grant permits

$$A_j = A_{j-1} \cap P_j, \qquad A_n = A_0 \cap \bigcap_{j=1}^{n} P_j. \qquad (C3)$$

Proposition T1. If restrictions cannot be deleted or substituted, the root grant is valid, and every step is evaluated in the same context, no descendant grant exceeds an ancestor grant.

Proof. Set intersection gives $A_j \cap P_j \subseteq A_j$. Induction yields $A_n \subseteq A_{n-1} \subseteq \cdots \subseteq A_0$. Constrained delegation in Macaroons embodies this established attenuation idea [6]. C3 is a set semantics, not a new cryptographic security proof.

Boundary. Grants recomputed at different times may expand or contract; the chain inclusion compares a fixed time and context. Translating permission to read temperatures within a household into a context-free read permission drops a restriction and violates the model. An intact signature chain cannot repair an incorrect context translation.

3.2 Multiple roots and authority splicing

A principal may receive independent grants from different owners. When policy permits alternative grant sources, a concrete action need only belong to one valid chain. However, a resource field from one credential must not be combined arbitrarily with an operation field from another. Each grant denotes a set of complete action tuples, not a list of independently composable fields.

For example, credential A allows reading resource A and credential B allows writing resource B. Their set union permits those two complete actions, not writing resource A. Taking a union of each field separately and then forming a Cartesian product can create actions absent from every original grant.

3.3 A composite task needs the union of action requirements

Suppose task T has action requirement sets $N_1, \ldots, N_k$. Under one root grant, a necessary authorization condition for completing all subtasks is

$$N_T = \bigcup_{i=1}^{k} N_i \subseteq A_0, \qquad N_i \subseteq A_i \subseteq A_0. \qquad (C4)$$

Proposition T2. If each subtask uses only actions in its grant A_i and C4 holds, the composition introduces no action outside the root grant.

Proof. Any action in the composition originates in a subtask and belongs to that subtask's A_i. Since $A_i \subseteq A_0$, it belongs to the root grant. This establishes only absence of authority escalation, not task completion.

Action requirements combine by union; delegation restrictions combine by intersection. These are different levels of reasoning. Intersecting every subtask's permissions can incorrectly reject a legitimate task, while freely unioning conditions from different roots can enlarge authority. A task with multiple roots needs one complete valid authorization path per action and must handle policies requiring joint approval. The existence of one authorizer does not satisfy a rule requiring several parties' consent.

4 Behavioral substitution and composition

4.1 Substitution must constrain behavior

Let o be an old capability and n its replacement. Sufficient conditions for preserving safety properties of callers relying on the old specification are

$$P_o \subseteq P_n, \qquad R_n(s) \subseteq R_o(s)\ (s \in P_o), \qquad E_n \subseteq E_o. \qquad (C5)$$

Proposition T3. If actual executions satisfy their specifications and C5 holds, the replacement produces neither a successor state forbidden by the old specification nor an extra effect from any

old valid pre-state.

Proof. Take $s \in P_o$. The first condition makes it a valid pre-state for the replacement. Every successor s' produced by the new capability belongs to $R_o(s)$ by the second condition. The third condition constrains effects. Any safety assertion holding for all successors allowed by the old relation therefore holds for successors of the new capability.

This uses established behavioral refinement and subtyping ideas [4]. Matching output types alone does not establish C5. Two capabilities may both return success, while one additionally unlocks a door. They have the same output structure but different allowed effects. C5 still does not establish termination or preservation of probability distributions, elapsed time, or resource consumption. Such properties must enter the specification before preservation can be analyzed.

4.2 Sequential composition

Consider capabilities c_1, c_2. If both satisfy partial correctness and $Q_1 \subseteq P_2$, then

$$R_1(P_1) \subseteq Q_1 \subseteq P_2, \qquad (R_2 \circ R_1)(P_1) \subseteq Q_2. \qquad (C6)$$

Proof. Starting from any P_1 state, the first capability, if it terminates, reaches Q_1 and hence satisfies the second capability's precondition. If the second capability terminates, its successor lies in Q_2. This is the Hoare-style sequential composition rule [5]. If an environmental transition R_e can occur between the capabilities, the linking condition must instead be $R_e(Q_1) \subseteq P_2$. Intervening state changes cannot be ignored.

An acyclic dependency graph supplies an execution partial order but proves no inclusion $Q_i \subseteq P_j$. Obtaining a temperature reading does not establish that the reading is still within a controller's accuracy and age requirements. Explicit error and validity conditions are needed to connect observation to control.

4.3 Noninterference in concurrent composition

Let $\mathcal{R}_i$ and $\mathcal{W}_i$ be the state components read and written by capability i. Assume the state is a product of these components, transitions depend only on declared read components and modify only declared write components, and there are no hidden shared resources or external interference. The following conditions suffice for exchanging the order of two actions:

$$\mathcal{W}_i \cap (\mathcal{R}_j \cup \mathcal{W}_j) = \varnothing, \qquad \mathcal{W}_j \cap (\mathcal{R}_i \cup \mathcal{W}_i) = \varnothing. \qquad (C7)$$

Proof argument. Neither action changes components read or written by the other. The second action therefore sees the same relevant input, and their writes do not overwrite each other. Their reachable-successor relations commute in this product-state model. If every unordered pair in a task partial order satisfies these conditions, any two topological orders can be related by swaps of adjacent unordered actions, preserving relational semantics.

Different device names do not imply physical noninterference. A heater and an air conditioner share room temperature, and robots may share a passage. Omitting such components makes a superficial check of C7 meaningless. A task graph also provides neither atomicity nor compensability. Physical effects that have already occurred generally cannot be reversed by a withdrawal declaration.

5 Revocation and temporal validity

5.1 The interval between authorization and commitment

Suppose an authority revokes a grant at real time t_r. Relevant decision makers learn of revocation within delay d. Credentials have maximum lifetime L, and the issuer's and decision maker's clocks each differ from real time by at most ϵ. An accepted action commits its effect within at most Δ. Assume no new credential for the grant is issued after revocation, credential lifetime cannot be extended, and expired credentials or credentials known to be revoked are no longer accepted.

Proposition T4. Under these assumptions, the window in which an old grant can still cause a commitment after revocation is bounded by

$$W \le \min\{d, L + 2\epsilon\} + \Delta. \qquad (C8)$$

Proof. Revocation propagation stops new acceptance of the old grant no later than $t_r + d$. Every credential issued before revocation has a latest real-time acceptance deadline no later than $t_r + L + 2\epsilon$. The two clock errors correspond to a fast issuing clock and a slow checking clock. Either stopping condition suffices, so the last acceptance is bounded by the earlier one. Already accepted actions may commit up to Δ later, yielding C8.

Checking validity again at commitment can tighten the bound. If accepted actions have no bounded commitment delay, C8 gives no finite bound. During a partition, d may be unbounded, but credential lifetime can still bound new acceptance under trusted-clock and no-reissuance assumptions. Withdrawing only a directory entry, without revoking action authority, does not establish C8's premises.

For example, $d = 5$ seconds, $L = 30$ seconds, $\epsilon = 0.1$ seconds, and $\Delta = 0.5$ seconds give a 5.5-second bound. During a partition the lifetime-based bound is 30.7 seconds. These are parameterized upper-bound examples, not measured revocation delays.

5.2 Immediate revocation conflicts with partition availability

Proposition T5. If an authority and an executor may remain disconnected indefinitely, and the executor receives no new information distinguishing remote revocation from nonrevocation, it cannot guarantee both no new action commitment after revocation and successful execution of every legitimate request during a partition when no revocation occurred.

Proof. Construct two worlds with identical histories at the executor. In one, the remote authority revokes the grant after the partition; in the other, it does not. Submit the same action request after the revocation time. Because the executor cannot observe the remote event, its decision, or randomized decision distribution, is identical in both worlds. Guaranteed execution in the second world implies execution in the first, violating immediate revocation. Refusal or indefinite waiting to ensure safety in the first world violates full authorization availability in the second.

This local indistinguishability argument relates to classical partition-consistency reasoning [7]. Availability here specifically means successful execution of legitimate actions, not merely returning an error response. Immediate revocation requires revocation and commitment to participate in a common, comparable, nonbypassable authoritative order, or requires stopping acceptance when evidence cannot be obtained. Increasing version numbers prevents rollback of known history; it does not prove that the executor knows the latest revocation.

6 From observational evidence to goal claims

6.1 Levels of authenticity and truth

Message signatures support judgments about the origin and integrity of selected message components [8]. A provider's signed task-completed statement protects the statement's origin and content, but leaves open provider error, sensor bias, and incorrect goal definitions. Evidence must distinguish who generated a message, whether a record changed, whether a measurement is reliable, and whether the goal actually holds.

Let G denote actual goal satisfaction and V a verifier's pass result. Write $\pi = \Pr(G)$ for the prior, $s = \Pr(V \mid G)$ for sensitivity, and $f = \Pr(V \mid \neg G)$ for the false-pass rate. For a positive denominator, Bayes' rule gives

$$\Pr(G \mid V) = \frac{s\pi}{s\pi + f(1-\pi)}. \qquad (C9)$$

For example, $\pi = 0.1, s = 0.95, f = 0.05$ yields a probability of actual goal satisfaction after a pass of approximately 0.679, not 0.95. This is a classical probability identity. Its role here is to prevent substitution among verifier accuracy, task base rates, and confidence after a pass. Without reliable parameter estimates, evidence should be reported as uncalibrated rather than assigned an invented trust probability.

6.2 Correlation among verifiers

Suppose two verifiers have false-pass rates at most α_1, α_2 when the goal is false. Without independence, one can still assert

$$\Pr(V_1 \cap V_2 \mid \neg G) \leq \min\{\alpha_1, \alpha_2\}. \qquad (C10)$$

Multiplication requires conditional independence given goal falsity and the relevant context. Two verifiers copying the same faulty sensor can have perfectly correlated errors. Each false-pass rate may be 0.05 while the joint rate remains 0.05, not 0.0025. More verifiers do not necessarily provide more independent evidence. Common data sources, model origins, and failure causes must be traced.

6.3 Conservative claims for multiple goals

Let the composite goal be $G = \bigcap_i G_i$, and let $\mathcal{E}$ be all accepted evidence. If each subgoal has a calibrated failure bound under that same complete evidence, $\Pr(\neg G_i \mid \mathcal{E}) \le \delta_i$, then

$$\Pr(G \mid \mathcal{E}) \ge \max\left\{0, 1 - \sum_i \delta_i\right\}. \qquad (C11)$$

Proof. Failure of G is the union of subgoal failure events. Apply the union bound in the common conditional probability space. No independence between goals is needed, but the bound may be conservative. Failure rates conditioned only on separate local evidence cannot automatically serve as the δ_i conditioned on all joint evidence. Additional evidence can change dependencies and posterior distributions.

If a subgoal has no failure bound, 1 is its only worst-case bound, potentially reducing the conjunction's probability guarantee to 0. Supported subgoal claims should remain available while the overall goal remains unresolved. Averaging subgoal confidence and declaring success is not justified.

7 Evidence dependencies and unresolved obligations

7.1 Dependency graphs and invalidation

Organize evidence and conclusions as a directed acyclic graph $G_E = (N_E, E_E)$, with edges from premises to dependent conclusions. Leaf evidence has an explicit source, time, scope, and acceptance condition. A derivation requires every listed premise and its inference rule to be valid. Alternative proof paths should be represented separately rather than mixed into an ambiguous premise list.

If evidence node z becomes invalid, let $\mathrm{Desc}(z)$ be the conclusions reachable along dependency edges. The scope that must be reconsidered is

$$\mathcal{I}(z) = \{z\} \cup \mathrm{Desc}(z). \qquad (C12)$$

Proposition T6. If conclusions depend only on their recorded premises and support is conjunctive, propagating invalidation along successors identifies all conclusions losing support from that proof. Conclusions outside the reachable set do not lose their graph-internal support because of this premise change.

Proof. Proceed in topological order. Direct dependents of z first lose a necessary premise, and their successors subsequently lose necessary support. This reaches every node on a dependency path. An unreachable node has no z in its transitive premise closure, so its support check does not involve z. Another independent proof may restore support. Loss of proof is not proof of falsity; the conclusion should become unresolved rather than automatically false.

Complete dependency recording is essential. Hidden shared sensors, common model errors, or omitted authority dependencies can make the graph underestimate the affected scope. Circular self-endorsement does not create independent evidence. A cycle without an external trusted basis cannot establish a claim through repeated citation.

7.2 From discovery coverage to an obligation frontier

The RCAP companion paper represents unsearched regions as an unresolved frontier. We propose a corresponding frontier of task obligations: necessary conditions in the task dependency graph that lack valid support. These can include unverified contracts, expired permissions, unfinished actions, and unconfirmed goal states. Result claims should follow from the supported subgraph and must not include goals still dependent on this frontier.

The two frontiers are not interchangeable. Counting bounds in discovery can establish candidate recall. In a task, one unresolved critical safety condition may block the whole task even when another 99 conditions are satisfied. Completing 99% of obligations generally does not imply a 99% success probability. Probabilistic claims require an explicit model and the calibration conditions of C11.

7.3 A running coordination example

Consider the goal of confirming that a room is empty, bringing its temperature into a specified interval, and confirming the result. Discovery locates occupancy detection, temperature measurement, and temperature control. The occupancy observation must meet an age requirement, control authority must cover the room, action scope, and time period, and temperature observations need an error bound. If a reading is y with absolute error at most η, a sufficient condition for the true temperature to lie in $[l, u]$ is

$$[y - \eta, y + \eta] \subseteq [l, u]. \qquad (C13)$$

For a target interval of 20 to 24 degrees, a reading of 23.8 with error bound 0.5 does not establish the goal. A reading of 22 with the same bound satisfies the condition at the measurement time. Guaranteeing that the goal remains satisfied for ten minutes requires assumptions on environmental change and subsequent control; one measurement is insufficient.

A person may enter after the occupancy check, so sequential composition must account for R_e. Completion of detection does not itself preserve the control action's precondition. If the occupancy evidence is later invalidated, dependent safety permission must be reconsidered, but an already completed temperature measurement does not thereby become false data. This

example places authority, time validity, behavior, and outcome evidence in one causal structure while showing why they cannot substitute for each other.

8 Proposed contribution and falsification criteria

The candidate contribution binds authority paths, relational contracts, and evidence dependencies to concrete task obligations, yielding result semantics with local support, local invalidation, and persistent unresolved states. It is not a universal trust score or an automatic correctness proof for every agent. Established work supplies the principal logical tools [3–8]. The framework's additional research value depends on the following questions.

Candidate claim	Checkable content	Explicit failure case
Authority paths prevent escalation through coordination	Trace a complete grant tuple for every actual action	An action is authorized only by splicing fields across credentials
Relational contracts constrain substitution beyond output types	Check successor and effect inclusion	A conforming output accompanies an effect forbidden by the old contract
Timing parameters bound the revocation window	Compare the latest commitment under every C8 premise	A commitment exceeds the bound without a violated assumption
Dependency graphs support local claim updates	Compare the complete dependency closure with affected claims	A hidden premise causes a required update to be missed
Unresolved obligations reduce unsupported overall success claims	Compare with semantics containing only a single success status	Overall verification is asserted while a necessary goal remains unresolved

Further work must address predicate expressiveness versus decidability, minimal disclosure of evidence across domains, physical-state abstraction errors, compromised trust roots, and calibration failure under shifting task distributions. This paper has not completed model checking, empirical comparisons, or a general proof of novelty. It does provide conditional arguments for its propositions. Stronger capabilities require stronger premises or new evidence, rather than broader wording.

9 Conclusion

Trustworthy capability coordination rests on traceable authority, composable behavioral specifications, explicit temporal conditions, and evidence tied to goals. Intersection along delegation paths constrains authority growth, while union of task action requirements expresses

actual demand. Relational contracts explain substitution and composition, and evidence dependencies determine which claims survive changes to premises. Keeping unresolved obligations visible allows local success and overall incompleteness to be expressed accurately at the same time.

References and verification notes

Sources were checked on 28 October 2026 through official publication pages, author institutional pages, or original papers. The stated role of each reference limits its evidential scope. Living specifications carry access dates, and classical references are not used as proof of originality for the joint framework.

[1] Model Context Protocol. Specification, 2026-07-28. Official version. Background on tool interaction.

[2] A2A Project. Agent2Agent Protocol Specification. Official specification. Living specification accessed 2026-10-28, replacing the previous repository-internal link.

[3] Rose, S.; Borchert, O.; Mitchell, S.; Connelly, S. Zero Trust Architecture. NIST SP 800-207, 2020. NIST publication. Access-control principles, not proof of physical goal satisfaction.

[4] Liskov, B. H.; Wing, J. M. A Behavioral Notion of Subtyping. ACM Transactions on Programming Languages and Systems, 16(6), 1994, 1811–1841. DOI: 10.1145/197320.197383. Authors' institutional paper. Foundation for behavioral substitution.

[5] Hoare, C. A. R. An Axiomatic Basis for Computer Programming. Communications of the ACM, 12(10), 1969, 576–580, 583. Oxford author bibliography. Foundation for sequential composition and partial correctness; title, year, and pages verified through the author's institutional bibliography.

[6] Birgisson, A.; Politz, J. G.; Erlingsson, Ú.; Taly, A.; Vrable, M.; Lentczner, M. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. NDSS, 2014. Conference publication. Background on constrained delegation.

[7] Gilbert, S.; Lynch, N. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. SIGACT News, 33(2), 2002. DOI: 10.1145/564585.564601. Authors' institutional paper. Related theory of partitions and indistinguishable executions.

[8] Backman, A.; Richer, J.; Sporny, M. HTTP Message Signatures. RFC 9421, 2024. RFC Editor. Normative basis for message origin and integrity, not evidence of real-world effects.

OpenTelemetry and general transport descriptions no longer support the trust argument and their references have been removed. Service-discovery background is concentrated in the RCAP companion paper rather than repeated here.